Разработка модулей ядра Linux: Часть 8. Интерфейсы модуля для взаимодействия с ядром

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

17.05.2012

Эта статья продолжает цикл статей, посвященный написанию модулей ядра Linux. В статье будут рассмотрены схемы взаимодействия и интерфейсы, по которым модуль взаимодействует с ядром операционной системы Linux.

Введение

В предыдущих статьях были подробно изучены отличия между программированием пользовательских процессов и модулей ядра. В данной статье будут рассмотрены схемы взаимодействия и интерфейсы, по которым модуль «сотрудничает» с другими компонентами операционной системы Linux.

Модуль ядра является некоторым связующим элементом для задач, которые выполняются в пространстве пользователя, с функциональностью, необходимой этим задачам и расположенной в пространстве ядра. Код модуля может использовать набор предоставляемых интерфейсов как для взаимодействия с монолитным ядром Linux (с кодом ядра, API ядра, структуры данных...), так и для взаимодействия с пользовательским пространством. Удобнее всего рассматривать механизмы коммуникации модуля в направлении пользователя и в направлении ядра по **отдельности**.

Взаимодействие модуля с ядром

Ядро (и ранее подгруженные к ядру модули) экспортируют набор имён, которые новый модуль использует при взаимодействии с ядром в качестве **API ядра**, о чём уже говорилось раньше. **Все** имена ядра содержатся в текстовом псевдофайле **/proc/kallsyms**. В ОС UNIX многие сущности, по возможности, представляется в виде файлов, а из файлов предпочтение отдаётся текстовым форматам.

© Copyright IBM Corporation 2012

Листинг 1. Фрагмент таблицы имён ядра

```
$ awk '/T/ && /print/ { print $0 }' /proc/kallsyms
...
c042666a T printk
...
c04e5b0a T sprintf
c04e5b2a T vsprintf
...
d087197e T scsi_print_status    [scsi_mod]
...
```

Вызовы API ядра осуществляются по прямому **абсолютному** адресу. С каждым именем, экспортированным ядром или любым его модулем, соотносится адрес, который и используется для связывания, при загрузке модуля, который захочет использовать это имя. Это основной механизм взаимодействия модуля с ядром.

Список имён ядра в **/proc/kallsyms** формируется динамически при каждой загрузке операционной системы. Изменения, вносимые в состав системы, (установка нового оборудования, драйверов, целевых модулей) будут изменять и содержимое этой таблицы. Кроме того, в ней динамически отслеживаются изменения в ходе работы уже загруженной системы, например, в процессе экспериментирования с модулями ядра. Число строк (имён) этой таблицы порядка 100000 (в зависимости от версии ядра и состава системы), но далеко не все имена из неё (порядка 10%) экспортируются и доступны для использования во внешних модулях. Эта таблица — один из основных источников, с которым постоянно работает программист модулей. Каждому имени ядра в **/proc/kallsyms** сопоставлено его функциональное назначение, указываемое односимвольным индикатором, который располагается перед именем. Ключ к его возможным значениям можно почерпнуть из справочника **man** по утилите **nm** (утилита анализа символов объектного формата), как показано в листинге 2.

Листинг 2. Литеры функционального назначения имён

```
$ man nm
...
    если литера на верхнем регистре, то символ глобальный (external).
...
    "D" Символ в инициализированной секции данных (data).
    "R" Символ в секции данных (data) только для чтения.
    "T" Символ в секции исполнимого кода (text).
```

Стоит перечислить особенности, общие для всех функций API ядра, которые обеспечивают модулю интерфейс для использования всех доступных возможностей ядра.

1. Эти функции реализованы в ядре, и даже при совпадении по форме с вызовами стандартной библиотеки языка C (например, вызов **sprintf()**) - это совершенно другие функции. Не стоит обманываться внешней похожестью или «почти похожестью», даже при сходной функциональности они могут отличаться «трудноуловимыми» деталями реализации. Заголовочные файлы для функций пространства пользователя располагаются в **/usr/include**, а для API ядра — в совершенно другом месте, в каталоге **/lib/modules/`uname -r`/build/include**.

2. Разработчики ядра Linux не связаны требованиями совместимости снизу вверх, в отличие от очень жёстких ограничений для пользовательских API, налагаемых стандартом POSIX. Поэтому API ядра может изменяться даже между **подверсиями** ядра. Функции ядра довольно плохо документированы (по крайней мере, в сравнении с документацией POSIX-вызовов для пользовательского пространства). Источниками для изучения особенностей реализации функций API ядра могут служить (в порядке приоритета и полезности):
   a. Прототипы и определения из заголовочных файлов в дереве каталогов **/lib/modules/`uname -r`/build/include**.
   b. Комментарии в тех же заголовочных файлах.
   c. Файлы формата .txt из каталога **Documentation** в дереве исходных кодов ядра, если это дерево установлено в системе (при инсталляции системы оно не устанавливается и также не устанавливается из репозитариев дистрибутива). Но на самом деле — это не полноценная документация, а только заметки к реализации: иногда там можно найти весьма полезные мелочи, иногда — ничего.
   d. Программные файлы ядра (.c) из того же дерева исходных кодов ядра. Это самый достоверный источник информации, но добыть из него информацию не так и просто.

   Других источников информации по API ядра, которым можно было доверять, фактически не существует, так как все остальные источники или устаревшие или ошибочные и т.д.
3. Существует общее правило (которое, правда, не всегда соблюдается), что функции API ядра в случае ошибки выполнения возвращают **отрицательный** результат завершения (код ошибки). Положительные возвращаемые результаты в API ядра используются для возврата численных результатов, а возвращаемое нулевое значение — как логический признак в некоторых API. Если вспомнить соглашения POSIX API, то они радикально отличаются: зачастую POSIX-функции помещают **положительный** код ошибки в **errno**. В качестве отрицательного результата при возникновении ошибки API ядра возвращают **те же числовые коды** ошибок, что и POSIX, но со знаком минус. Этого же правила рекомендуют придерживаться и при написании собственных функций в теле модуля, в частности, **функции инициализации** модуля. Такие соглашения существуют в пространстве ядра.

Коды ошибок

Коды ошибок, возвращаемые функциями API ядра при возникновении ошибки в ходе выполнения, в основной массе совпадают с кодами ошибок (хотя есть и исключения), известным по API пространства пользователя. В ядре коды завершения определены в файлах **<asm-generic/errno.h>** и **<asm-generic/errno-base.h>** (в листинге 3 показан достаточно большой фрагмент таблицы, но именно эти коды будут активно использоваться в последующих примерах).

Листинг 3. Коды ошибок ядра (начало таблицы)

```
$ cat /lib/modules/`uname -r`/build/include/asm-generic/errno-base.h \
head -n20
#define EPERM            1      /* Операция не поддерживается */
#define ENOENT           2      /* Нет такого файла или каталога */
#define ESRCH            3      /* Нет такого процесса */
#define EINTR            4      /* Прерванный системный вызов */
#define EIO              5      /* Ошибка ввода/вывода */
#define ENXIO            6      /* Нет такого устройства или адреса */
#define E2BIG            7      /* Список аргументов слишком длинный */
#define ENOEXEC          8      /* Ошибка формата вызова exec */
#define EBADF            9      /* Ошибочный дескриптор файла */
#define ECHILD          10      /* Нет дочернего процесса */
#define EAGAIN          11      /* Повторите попытку снова */
#define ENOMEM          12      /* Недостаточно памяти */
#define EACCES          13      /* Доступ запрещён */
#define EFAULT          14      /* Неверный адрес */
#define ENOTBLK         15      /* Требуется блочное устройство */
#define EBUSY           16      /* Устройство или ресурс заняты */
```

Заключение

В этой части цикла было систематически проанализировано взаимодействие между модулем ядра и самим ядром. В следующей статье будут изучены интерфейсы, используемые для взаимодействия между модулем ядра и пространством пользователя.

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)